

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

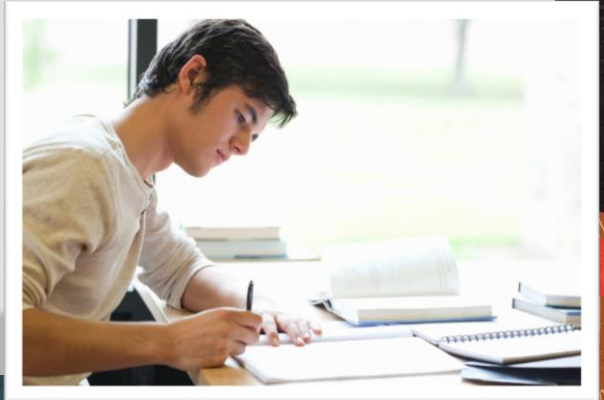
Academy

Database Foundations

2-1

Relational Databases

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Describe the features of a single table
 - Describe the features and rules of a relational database
 - Describe the advantages and disadvantages of databases types
 - Define relational tables and key terms



Single Table

- As discussed previously, a flat file database is a type of database that stores data in a single table
- Flat file databases are generally in plain-text form, where each line holds only one record

	CUSTOMER_ID	FIRST_NAME	LAST_NAME	ADDRESS	CITY
Record 1	CN0001	Rick	Carmen	King Street	Boston
Record 2	CN0002	Nick	Doris	6 Rotrat	Boston
Record 3	CN0003	Mary	Pink	Centrale	Lagos

Advantages and Disadvantages of a Flat File Database

Advantages

- Easy to understand
- Easy to implement
- Easy to extract information
- All records stored in one place
- Simple sorting and filtering of reports
- Less hardware and software requirements

Disadvantages

- Less security
- Data inconsistency
- Data redundancy
- Cumbersome sharing of information
- Slow for huge databases

A flat file database is simpler and less expensive to use. It fits the needs of many small businesses and home users.

A flat file has many limitations because it contains ALL data in ONE table.

Here are some **more** disadvantages of flat file databases:

- No support for multi-user access
- Limited to smaller databases
- Limited functionality
- Integrity issues
- No security
- No backup and recovery
- No protection for data in concurrent access

Example of a Flat File Database

	AUTHOR_ID	AUTHOR_NAME	TITLE
Record 1	AD0001	Oscar Wilde	A Vision
Record 2	AD0002	Leo Tolstoy	War and Peace
Record 3	AD0003	Oliver Goldsmith	Citizen of the World
Record 4	AD0003	Oliver Goldsmith	Androcles and the Lion

Notice that both the details for the books and authors are stored in this single table, and there is a repetition of values.

Relational Databases

- A relational database presents information in tables with rows and columns
- Each column represents a particular type of information (a field), and each row lists one record
- The tables are then related to one another by using a common field
- A unique field called a key is used to identify each record in a relational database



A relational database management system (RDBMS) stores data in tables. Each table is given a name by the user who creates the table. The user generally chooses a name that correlates to the data that will be stored in the table; for example, STUDENTS, EMPLOYEES, LOCATIONS. When a table is created, the user also creates and names columns related to the specific characteristics that are stored for each record.

Relational Database: Example

STUDENTS

ID	LAST_NAME	DATE_OF_BIRTH	ADDRESS	COURSE_ID

Foreign Key

Primary Key

Relationship

ID	NAME	DURATION

COURSES

Each table is assigned a `PRIMARY_KEY` column which uniquely identifies the entity instance

A `PRIMARY_KEY` column in one table is designated as a `FOREIGN_KEY` column in a related table to form a relationship between the tables

This relationship between the `STUDENTS` table and the `COURSES` table lets you store the data and query it to determine the specific courses that a student is attending (or has attended)

Advantages of a Relational (Multiple-Table) Database

- Less redundancy
- Avoidance of inconsistency
- Efficiency
- Data integrity
- Confidentiality

*See Notes for more detail

Compared to the flat file system, the RDBMS has several advantages:

- **Less redundancy:** In a flat file system, there is much redundancy. For example, the author names are stored multiple times.
- **Avoidance of inconsistency :** If the same piece of information is stored in more than one place, then any changes in the data need to be made in all places where the data is stored.
- **Efficiency:** A database is usually more efficient than a flat file system, because a piece of information is stored in fewer locations.
- **Data integrity:** In a database system, it is easier to maintain data integrity because strong data types are assigned to each column.
- **Confidentiality:** It is easier to maintain the confidentiality of the information if data storage is centralized in one location.

Relational Tables

- A table is a simple structure where data is organized and stored

Table: EMPLOYEES

columns					
EMPLOYEE_ID	LAST_NAME	FIRST_NAME	DEPARTMENT_ID	PAYROLL_ID	NICKNAME
100	SMITH	DANA	10	21215	Dana
310	ADAMS	TYLER	15	59877	Ty
210	CHEN	LAWRENCE	10	1101	Larry
405	GOMEZ	CARLOS	10	52	Chaz
378	LOUNGANI	NEIL	22	90386	Neil

rows

Primary Key Column (PK)

Foreign Key Column (FK)

Unique Key Column (UK)

**** Note :** types of keys shown here will be discussed later in the lesson and course

ORACLE
Academy

DFo 2-1
Relational Databases

Copyright © 2020, Oracle and/or its affiliates. All rights reserved. 10

The PAYROLL_ID column is a unique key. This means that the system does not allow two rows with the same payroll_id.

In this example, the department_id foreign key refers to a row in the DEPARTMENTS table. You know that Dana Smith works in department 10. If you wanted to know more about Dana Smith's department, you would look for the department_id = 10 row in the DEPARTMENTS table.

Rules for Relational Database Tables

- Each table has a distinct name
- Each table may contain multiple rows
- Each table has a value to uniquely identify the rows
- Each column in a table has a unique name
- Entries in columns are single values
- Entries in columns are of the same kind
- Order of rows and columns is insignificant



Key Terms

- **Table** – A basic storage structure
- **Column** – attribute that describes the information in the table
- **Primary Key** – the unique identifier for each row
- **Foreign Key** – a column that refers to a primary key column in another table
- **Row** – data for one table instance
- **Field** – the one value found at the intersection of a row and column

Summary

- In this lesson, you should have learned how to:
 - Describe the features of a single table
 - Describe the features and rules of a relational database
 - Describe the advantages and disadvantages of databases types
 - Define relational tables and key terms



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Foundations

2-2

Conceptual and Physical Data Models

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Describe a conceptual data model
 - Describe a logical data model
 - Describe a physical data model
 - Analyze the similarities and differences between conceptual and physical data models



What Is a Conceptual Model?

- Captures the functional and informational needs of a business
- Is based on current needs, but may reflect future needs
- Addresses the needs of a business (what is conceptually ideal), but does not address its implementation (what is physically possible)

What Is a Conceptual Model?

- Identifies :
 - important entities (objects that become tables in database)
 - relationships among entities
- Does not specify :
 - attributes (objects that become columns or fields in database)
 - unique identifiers (attribute that becomes primary key in database)

What Is a Logical Model?

- Includes all entities and relationships among them
- Is called an entity relationship model (ERM)
- Is illustrated in an ERD
- Specifies all attributes and UIDs for each entity
- Determines attribute optionality
- Determines relationship optionality and cardinality *

A logical data model describes the data in as much detail as possible, without regard to how it will be physically implemented in the database. It is normally derived from a conceptual data model.

* We will discuss optionality and cardinality later in the course

What Is a Physical Model?

- Is an extension to a logical data model
 - Defines table definitions, data types, and precision
 - Identifies views, indexes, and other database objects *
- Describes how the objects should be implemented in specific database
- Shows all table structures, including columns, primary keys, and foreign keys

Physical modeling deals with the conversion of the logical data model to a relational database model. Each relational model can have one or more physical models, one for each RDBMS that you deploy to.

* We will discuss views, indexes and other database objects later in the course.

Steps to Create a Physical Data Model

**Model
entities as
tables**

**Model
relationships
as foreign keys**

**Model
attributes as
columns**

**Modify the
physical data
model based on
physical
constraints and
requirements**

Conceptual and Physical Models

- The art of planning, developing, and communicating produces a desired outcome
- Data modeling is the process of capturing the important concepts and rules that shape a business and depicting them visually in a diagram
- This diagram becomes the blueprint for designing the physical thing
- The client's dream (conceptual model) becomes a physical reality (physical model)

The conceptual model is concerned with the real-world view and understanding of data. The physical model specifies how it will be executed in a particular database management instance.

Conceptual/Logical Model: Case Scenario



Faculty

Matt, I would like you to create a simplified library database to store the book details in our department

Sure. I will start by identifying the entities and attributes and their relationships



Matt

Case Scenario: Creating a Conceptual Model

- A Conceptual data model documents important entities and how they relate to each other

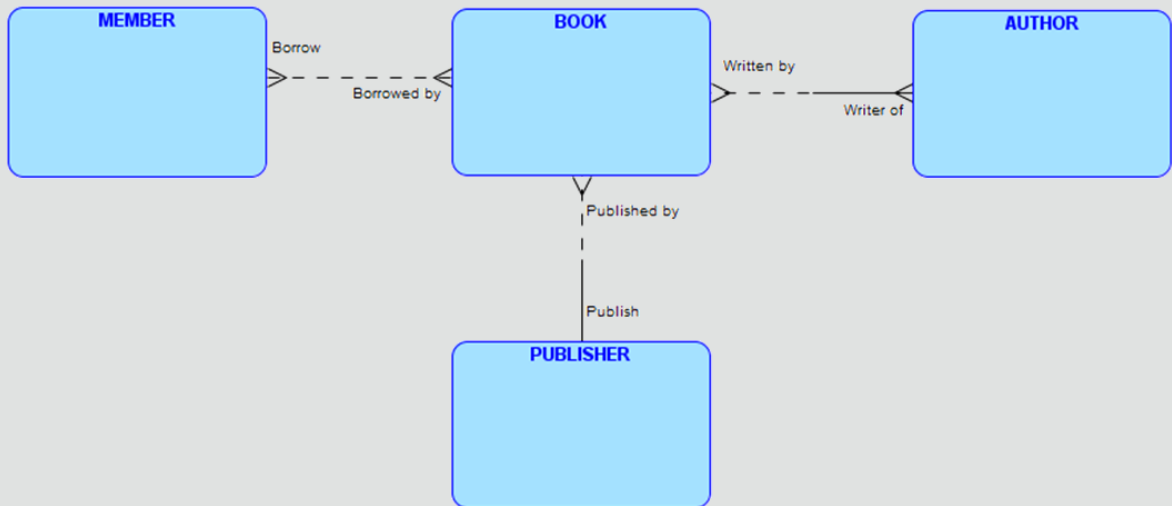


Diagram was created using Oracle SQL Data Modeling tool which will be covered in Section 4 of course material.

Case Scenario: Creating a Logical Model

- A logical data model documents the information requirements of the business

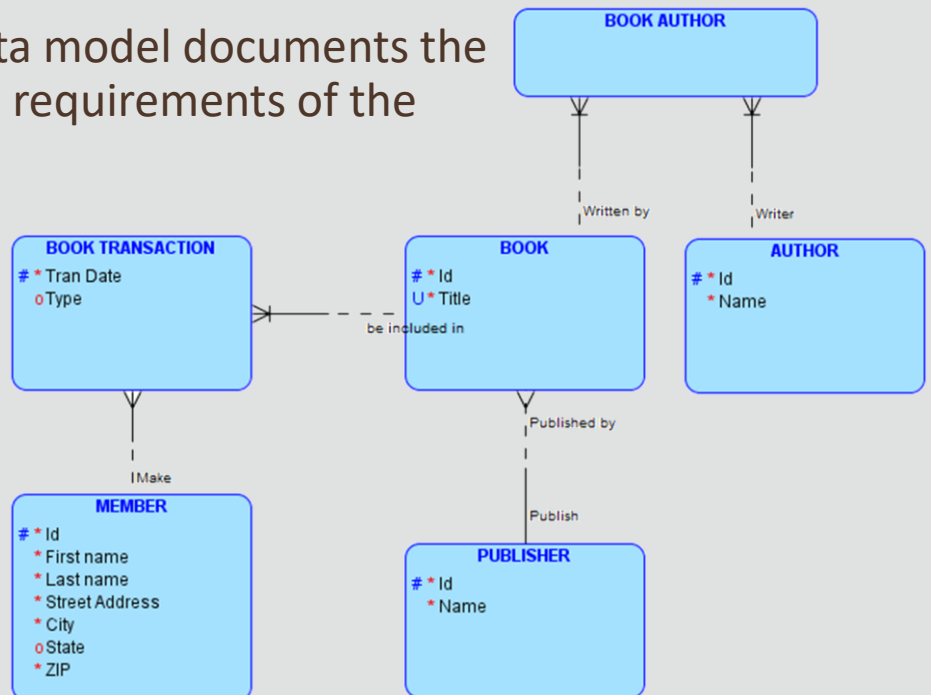


Diagram was created using Oracle SQL Data Modeling tool which will be covered in Section 4 of course material.

Physical Model: Case Scenario



Faculty

Matt, I want to know the specifications for all tables and columns required in the simplified library database

Sure. I will convert the entities and their attributes into tables and columns

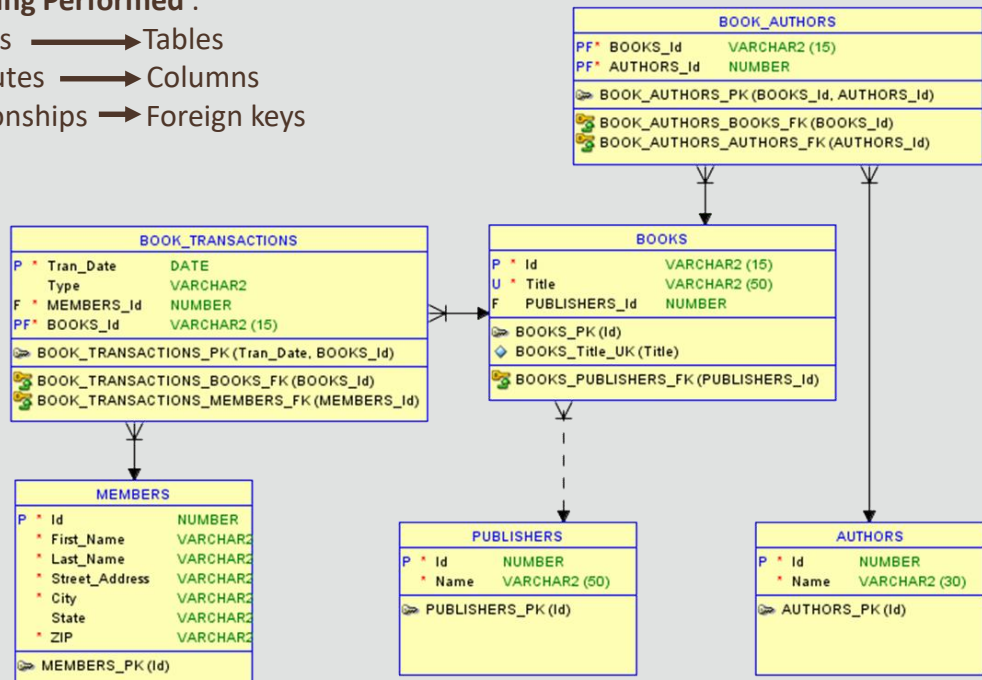


Matt

Case Scenario: Creating a Physical Model

Modeling Performed :

Entities —————> Tables
 Attributes —————> Columns
 Relationships —> Foreign keys



ORACLE
Academy

DFo 2-2
Conceptual and Physical Data Models

Copyright © 2020, Oracle and/or its affiliates. All rights reserved. 14

Diagram was created using Oracle SQL Data Modeling tool which will be covered in Section 4 of course material.

Summary

- In this lesson, you should have learned how to:
 - Describe a conceptual model
 - Describe a logical model
 - Describe a physical model
 - Analyze the similarities and differences between conceptual and physical data models



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Foundations

2-3

Entities and Attributes

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Identify entities
 - Identify attributes
 - Identify mandatory, optional, volatile, and nonvolatile attributes
 - Describe the Barker, Bachman, and Information Engineering notations



Entity

- Information that must be tracked
- Name for things that you can list (usually in noun form)



EMPLOYEE



DEPARTMENT

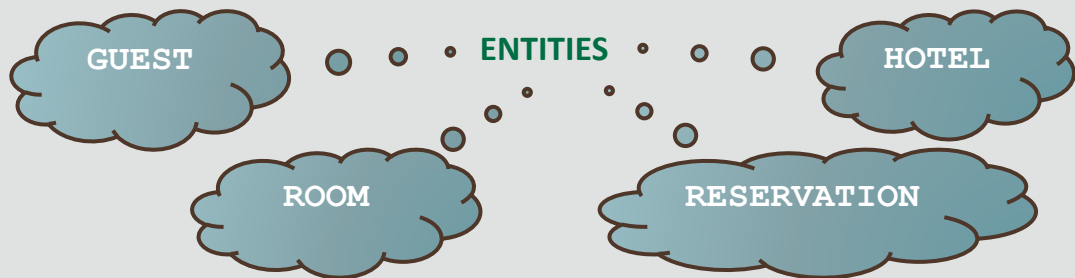
Entities are categories of things that are important for a business and about which information must be kept. Entities contain facts and information that the business must know and remember. Some examples of entities might include the following:

- **PERSON:** Agent, insured, employee, customer
- **PLACE:** State, country, municipality
- **THING:** Inventory item, vehicle, product
- **CONCEPT:** Policy, risk, coverage, job
- **ORGANIZATION:** Agency, department
- **EVENT:** Service request, claim, election

Identifying Entities: Example



District Hotel Manager Sharon Ferguson wants to manage the information that her company keeps about their hotels, guests, and rooms. Sharon is responsible for multiple hotels in various locations. She wants to know what rooms she has in each hotel and what the price is for each room. She also wants to know which rooms have been reserved for a particular date by a guest.



Entity Types

- An entity can be classified as one of the following types:

Name	Description	Example
Prime	Exists independently	CUSTOMER, INSTRUCTOR
Characteristic	Exists because of another (prime) entity	ORDER, CLASS OFFERING
Intersection	Exists because of two or more entities	ORDER ITEM, CLASS ENROLLMENT

Entities and Instances

- Entities contain instances
- An entity instance is a single occurrence of an entity
- Entities represent a set of instances that are of interest to a particular business



Entity	Instance
PERSON	John Smith
PRODUCT	2.5 x 35 mm copper nail
PRODUCT TYPE	Nail
JOB	Violinist

An instance of one entity may be an entity in its own right. For example, the "violinist" instance of the **JOB** entity could be the name of another entity with instances like Tim Kary or Jerry Gam.

At the end of the entity modeling process, entities are typically transformed into tables, with each row of a table representing an individual instance.

Project Exercise 1

- DFo_2_3_1_Project
 - Oracle Baseball League Store Database
 - Identifying Entities



ORACLE
Academy

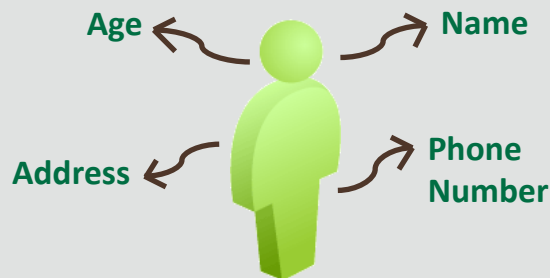
DFo 2-3
Entities and Attributes

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

8

Attributes

- Attributes describe entities and are the specific information that must be known
- It is a single-valued property detail of an entity



Attributes describe an entity by qualifying, identifying, classifying, quantifying, or expressing the state of the entity. Attributes represent a type of description or detail, not an instance.

Attribute names are singular. An attribute value can be a number, a character string, a date, or an image. Every attribute has a specific data type in the database.

Attribute Characteristics

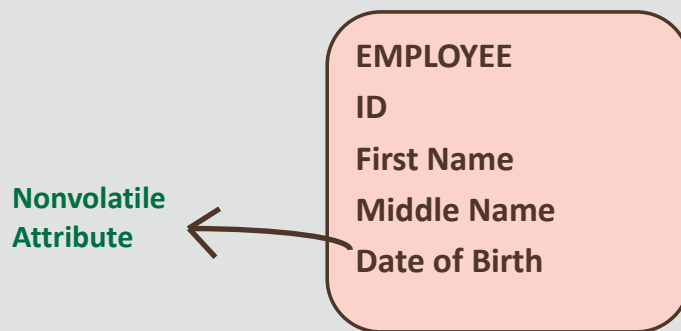
- Attributes are shown within the entity box on the ERD
- Attribute names are singular and mixed case or lowercase
- In most cases, the name of the attribute should not include the entity's name, because attributes are qualified with the entity name

Attribute Characteristics

- Attributes are classified as one of the following:
 - Mandatory (nulls are not allowed), indicated by *
 - Optional (nulls are allowed), indicated by a lowercase o

Volatile and Nonvolatile Attributes

- Volatile attributes are unstable attributes
 - Example: Age
- Nonvolatile attribute are stable attributes
 - Example: Birth Date



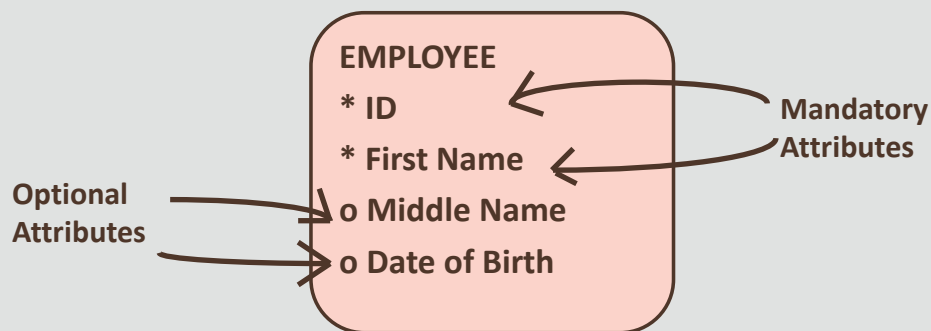
Project Exercise 2

- DFo_2_3_2_Project
 - Oracle Baseball League Store Database
 - Identifying Attributes



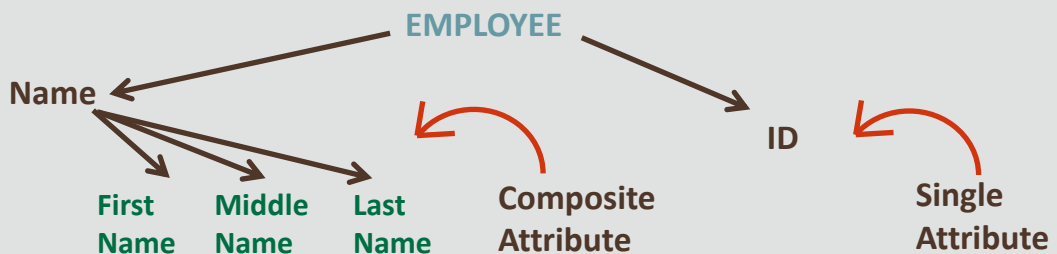
Mandatory and Optional Attributes

- Mandatory attributes must have a value
- Optional attributes may not have a value and can be blank (null)



Single and Composite Attributes

- Single or atomic attributes are attributes that cannot be divided into subparts
- Composite attributes are attributes that can be divided into smaller subparts that represent basic attributes with independent meanings of their own



Emp ID, age, and sex are examples of simple attributes.

Single-Valued and Multi-Valued Attributes

- Single-valued attributes can have a single value at a particular instance of time
 - Example: Student Last Name
- Multi-valued attributes can have more than one value at one time
 - Example: Address



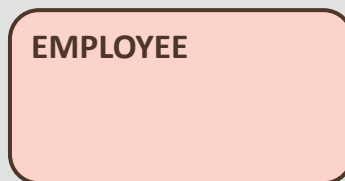
Project Exercise 3

- DFo_2_3_3_Project
 - Oracle Baseball League Store Database
 - Identifying Mandatory/Optional Attributes



Barker Notation: Rules for Drawing Entities

- An entity is represented as a round-cornered rectangle
- An entity must be named, and the name must be placed inside the entity in the upper-left corner
- The entity name should be in uppercase form
- The entity name should be in singular form



The Barker notation is used in this course.

Barker Notation: Rules for Drawing Attributes

- The attributes must be written so that everyone, not just developers, can understand them
- Attributes must be written with the first letter of each word in uppercase and the rest in lowercase
- A symbol representing the type of attribute should be placed next to each attribute (* for mandatory o for optional and # for UID)

EMPLOYEE

Id

* First Name

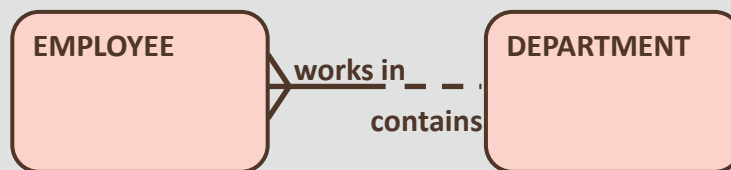
* Last Name

* Date of Birth

* Telephone Number

Barker Notation: Rules for Drawing Relationships

- A relationship can exist between a maximum of two entities
- A relationship can exist on the same entity (recursive)
- A relationship has two perspectives
- Both perspectives of a relationship must be labeled



Bachman Notation

- Entity (represented by a box)
- Attributes
- Relationship lines

EMPLOYEE
P Id
* First Name
* Last Name
* Date of Birth
Telephone Number



ORACLE
Academy

DFo 2-3
Entities and Attributes

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

21

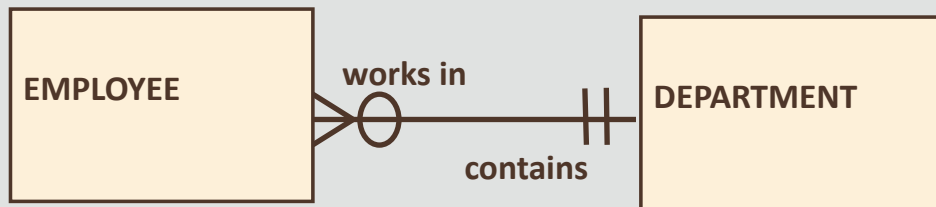
The Bachman notation is also a binary model.

- Attributes:
 - Show * for not null and no notation for null attributes.
 - Show P for UID and F for the attribute created through the relationship
- Relationship lines:
 - Show an arrow instead of crow's feet for the maximum cardinality
 - Show an open circle or filled-in circle instead of a dotted line and a solid line for the minimum cardinality

The relationship is depicted as a line connecting the associated entities. The relationship is given a phrase to describe the association at both ends of the line.

In Bachman notation, a subclass is represented as an inner box within the superclass.

Information Engineering Notation



An EMPLOYEE works only in one DEPARTMENT
A DEPARTMENT contains zero or more EMPLOYEES

zero or more ————

zero or one ————

one or more ————

only one ————

The Information Engineering notation is also a binary method. It is simple and easy to read, and is well suited for high-level logical and enterprise data modeling.

It does not allow a ternary relationship, and it does not show attributes related to a relationship. Both cardinality and participation constraints are combined into the min/max (bar and crowfoot) notation.

The only drawback of this notation is that it does not support identification of an entity's attributes.

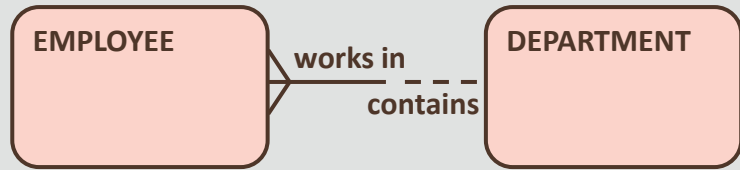
Data Model Notations

Notation (Read left to right)	Barker Notation	Bachman Notation	Information Engineering
Zero or one			
Only one			
Zero or more			
One or more			
Primary Key/Unique key	#	P	

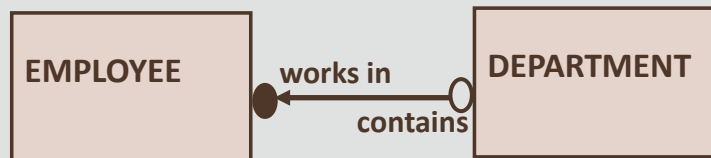
Note: Barker notation is used for this course

Data Model Notations: Examples

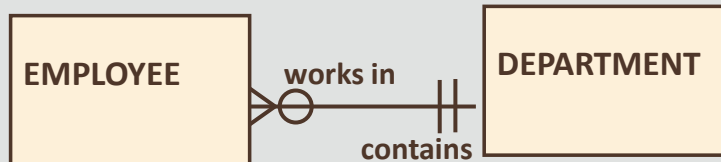
Barker Notation



Bachman Notation



Information Engineering Notation



Project Exercise 4

- DFo_2_3_4_Project
 - Oracle Baseball League Store Database
 - Using Barker Notation



Summary

- In this lesson, you should have learned how to:
 - Identify entities
 - Identify attributes
 - Identify mandatory, optional, volatile, and nonvolatile attributes
 - Describe the Barker, Bachman, and Information Engineering notations



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Foundations

2-4

Unique Identifiers

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Identify unique identifiers (UIDs)
 - Identify artificial unique identifiers
 - Identify composite unique identifiers
 - Identify candidate and secondary unique identifiers
 - Define primary keys

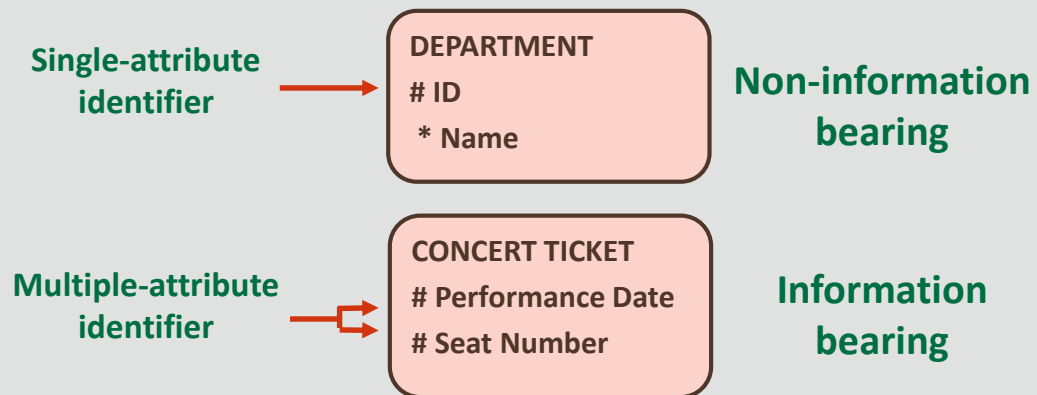


Unique Identifiers

- A unique identifier is an attribute of an entity that meets the following rules:
 - It is unique across all instances of the entity
 - It has a non-NULL value for each instance of the entity for the lifetime of the instance
 - It has a value that never changes for the lifetime of the instance
- A UID is a special attribute or group of attributes that uniquely identifies a particular instance of an entity

Unique Identifier: Examples

- Each entity must have a unique identifier
- Otherwise, it is not an entity



The UID can be information bearing, when its values have some business meaning. A non-information-bearing UID is sometimes referred to as a artificial key. An information-bearing UID is sometimes referred to as a natural key.

Simple UUIDs vs Composite UUIDs

- A UUID that is a single attribute is a simple UUID
- Sometimes a single attribute is not enough to uniquely identify an instance of an entity
- If the UUID is a combination of attributes its called a composite UUID

Simple UUID



CONCERT TICKET
Ticket Number
* Name

Composite UUID



CONCERT TICKET
Performance Date
Seat Number

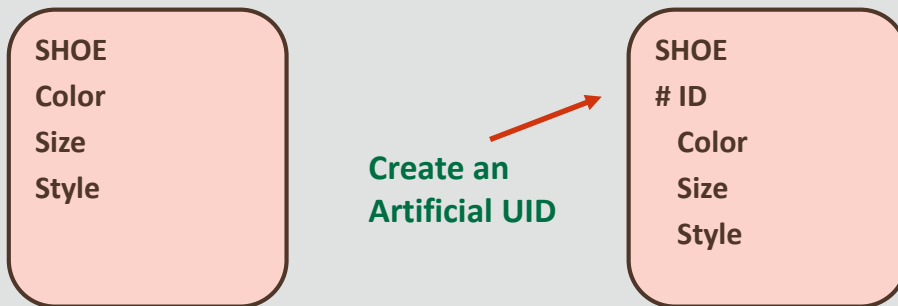
Project Exercise 1

- DFo_2_4_1_Project
 - Oracle Baseball League Store Database
 - Identifying Unique Identifiers (UIDs)



Artificial Unique Identifier

- An artificial UID is made from data that is assigned or generated by the system
- Artificial UIDs do not occur in the natural world but are created for identification purposes in a system



Project Exercise 2

- DFo_2_4_2_Project
 - Oracle Baseball League Store Database
 - Identifying Artificial Unique Identifiers



Case Scenario

MEMBER

First Name

Last Name

Street Address

How can you uniquely identify a member?

Can you use a combination of first name and last name? Only if you are sure that the combination is unique.

MEMBER

First Name

Last Name

Street Address

Case Scenario

MEMBER

ID

First Name

Last Name

Address

First Name and Last Name can be the same for more than one member. So, let's create an artificial UID called ID

Candidate Unique Identifiers

- An entity can have more than one UID
- Candidate UIDs:
 - Badge number
 - Payroll number
- Only one of the candidate UIDs can be chosen as the primary UID
- The other candidates are called secondary UIDs

EMPLOYEE

Badge Number
Payroll Number
First Name
Last Name

There is no standard diagramming convention for tagging secondary UIDs.

Case Scenario

MEMBER

ID

Email

First Name

Last Name

Street Address

ID has been chosen as the primary UID in the Member entity. Can you identify the candidate UID?

Case Scenario

Email can be chosen as the candidate UID because email is unique for every member.

MEMBER

ID
(#) Email
First Name
Last Name
Street Address

Primary Key

- A UID becomes a primary key when the logical model is transformed into a physical database
- A primary key (PK) is a column or set of columns that uniquely identifies each row in a table
- It cannot contain null values
- A PK is either an existing table column or a column that is specifically generated by the database according to a defined sequence
- It must contain a unique value for each row of data



Primary Key: Example

EMPLOYEES

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		DEPARTMENT_ID
100	Steven	King		90
101	Neena	Kochhar		90
201	Rick	Bel		90
205	Shelly	Higgins		10
300	Bill	Steveward		110

↑
Single-Column PK

Composite Primary Key: Example

ACCOUNTS

BANK_NO	ACCT_NO	BALANCE	DATE_OPENED
104	71432	12,000	29-Oct-90
104	34578	18,000	12-Sep-85
105	78967	20,000	
103	96545	60,000	04-Aug-91
105	72345	10,000	03-Jan-99



These fields are combined to form a “composite” PK – a PK consisting of multiple fields

Summary

- In this lesson, you should have learned how to:
 - Identify unique identifiers (UIDs)
 - Identify artificial unique identifiers
 - Identify composite unique identifiers
 - Identify candidate and secondary unique identifiers
 - Define primary keys



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Foundations

2-5

Relationships

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

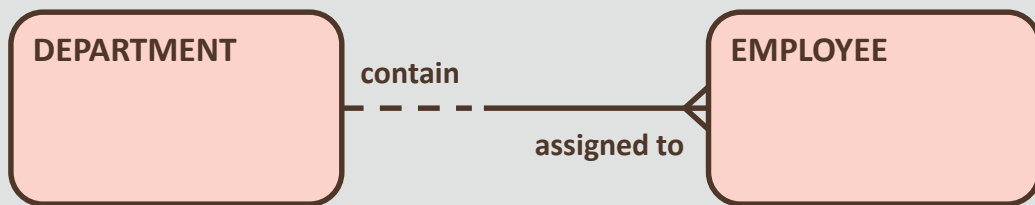
Objectives

- This lesson covers the following objectives:
 - Define and recognize examples of relationships and corresponding foreign keys
 - Identify the optionality of relationships
 - Identify the cardinality of relationships
 - Types of relationships
 - Relationship matrix



Relationships

- A relationship is a bidirectional, significant association between two entities or between an entity and itself



A relationship represents the business rules that link entities. Each relationship always has two business rules. In the slide example, the business rules are:

- A **DEPARTMENT** may contain one or many **EMPLOYEEs**.
- An **EMPLOYEE** must be assigned to one and only one **DEPARTMENT** .

Relationships

- Relationships represents an association between two or more entities
- The relationship line on the diagram is either solid (mandatory) or dashed (optional)
- These lines terminate in either a "single toe" (one instance) or a "crow's foot" (one or more instance)

In a conceptual data model, a relationship is any association, linkage, or connection between the entities of interest to the business.

Relationships

- Relationships have names that help describe the connection between the entities
- In the relationship diagram the name of the relationship, from either perspective, is printed near the starting point of the relationship line (see slide 5)
- Examples:
 - DEPARTMENTS contains EMPLOYEES
 - EMPLOYEES assigned to DEPARTMENTS

Foreign Key

- Relationships in a conceptual data model are mapped to foreign keys in a physical database table
- A foreign key (FK) is a column or a combination of columns in one table that refers to a primary key in the same table or another table

Foreign Key Examples

EMPLOYEES

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
100	Steven	King	90
101	Neena	Kochhar	90
102	Lex	De Haan	90
103	Alexander	Hunold	60
104	Bruce	Ernst	60

Foreign
Key

refers to

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME
10	Administration
20	Marketing
50	Shipping

Primary Key →

ORACLE
Academy

DFo 2-5
Relationships

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

8

To link these two tables and determine employee and department information, a corresponding DEPARTMENT_ID column must be inserted in the EMPLOYEES table to reference existing department IDs in the DEPARTMENTS table. In this case, the DEPARTMENT_ID column in the EMPLOYEES table is a foreign key that references a column that has the same name in the DEPARTMENT table. This is an example of a relationship between two tables.

Components of a Relationship

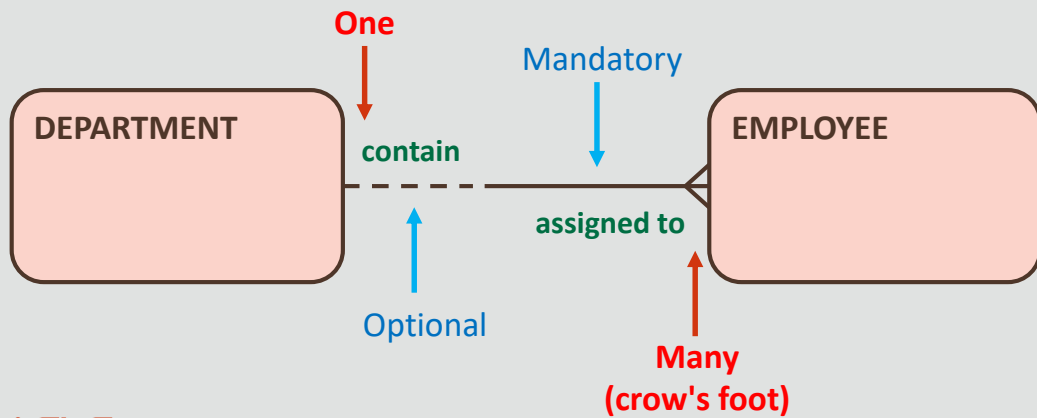
- The components of the relationship include the following:
 - Name: The label that appears close to the entity it is assigned to. Make sure that all relationship names are in lowercase
 - Cardinality: The minimum and maximum number of values in the relationship
 - One and only one matching record
 - One or more matching records
 - Optionality: Whether the relationship must exist
 - Optional (zero matching records)
 - Mandatory (at least one matching record in each entity)

Components of a Relationship

- When reading the business rule sentence, use the following wording:
 - Optional: Use "may be" or "may."
 - Mandatory: Use "must be" or "must."
 - Line: Use "one and only one."
 - Crow's feet: Use "one or more."
- Business rule syntax is as follows:
 - Each entity1 {must be or may be} relationship name
{one or more or one and only one} entity2

Components of a Relationship

- Name
- Cardinality
- Optionality



Project Exercise 1

- DFo_2_5_1_Project
 - Oracle Baseball League Store Database
 - Recognize Examples of Relationships



What Is Optionality in a Relationship?

- Relationships are either mandatory or optional
- Consider the two entities, EMPLOYEE and JOB
- Based on what you know about instances of the entities, you can determine optionality by answering two questions:
 - Must every employee have a job? In other words, is this a mandatory or an optional relationship for an employee?
 - Must every job be done by an employee? In other words, is this a mandatory or optional relationship for a job?

Project Exercise 2

- DFo_2_5_2_Project
 - Oracle Baseball League Store Database
 - Identify the Optionality of Relationships



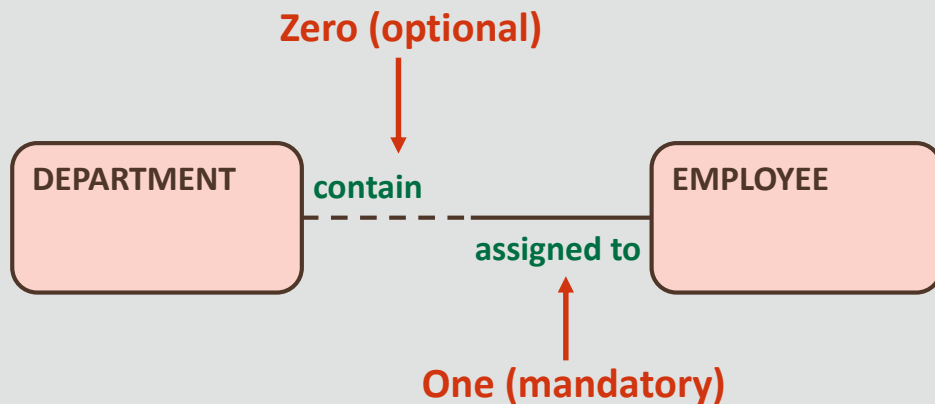
What Is Cardinality in a Relationship?

- Cardinality measures the quantity of something
- In a relationship, it determines the degree to which one entity is related to another by answering “How many?”
- For example:
 - How many jobs can one employee hold? One job only? Or more than one job?
 - How many employees can hold one specific job? One employee only? Or more than one employee?

The cardinality of a relationship answers only whether the number is singular or plural; it does not answer with a specific plural number.

Determining the Relationship's Minimum Cardinality

- What is the minimum cardinality in each direction?



The first question to answer is this: What is the minimum cardinality for each direction of the relationship?

In the slide example, answer the following questions:

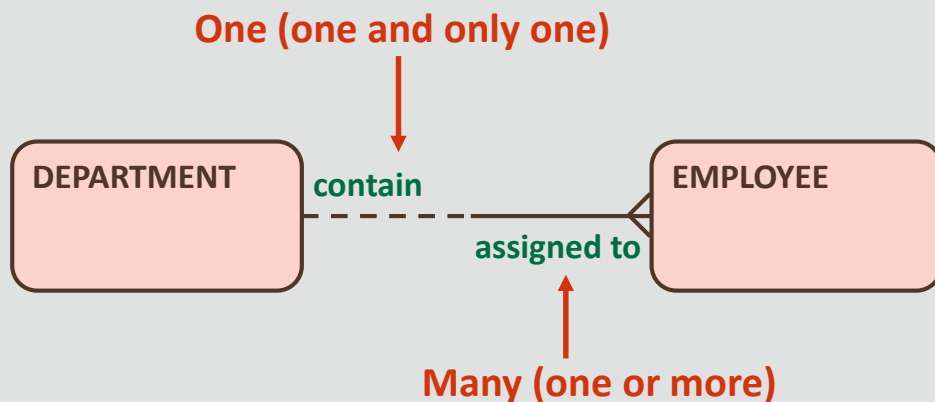
1. Must an EMPLOYEE be assigned to a DEPARTMENT? The answer is always.
2. Is there any situation in which an EMPLOYEE will not be assigned to a DEPARTMENT?
The answer is no. An EMPLOYEE must always be assigned to a DEPARTMENT. (Mandatory)
3. Must a DEPARTMENT be composed of an EMPLOYEE?
The answer is no. A DEPARTMENT does not have to be composed of an EMPLOYEE. (Optional)

When the minimum cardinality is optional, the value could be zero. When the minimum cardinality is mandatory, the value must be at least one.

Note that the relationship line in the slide was intentionally drawn without the maximum cardinality.

Determining the Relationship's Maximum Cardinality

- What is the maximum cardinality in each direction?



The second question to answer is this: What is the maximum cardinality for each direction of the relationship?

In the slide example, answer the following questions:

1. Must an EMPLOYEE be assigned to more than one DEPARTMENT?
The answer is no. An EMPLOYEE must always be assigned to one and only one DEPARTMENT. (One)
2. May a DEPARTMENT be composed of more than one EMPLOYEE?
The answer is yes. A DEPARTMENT may be composed of one or more EMPLOYEEs. (Many)

When the maximum cardinality is one, the value can be only one. When the maximum cardinality is many, the value can be one or more.

Project Exercise 3

- DFo_2_5_3_Project
 - Oracle Baseball League Store Database
 - Identify the Cardinality of Relationships

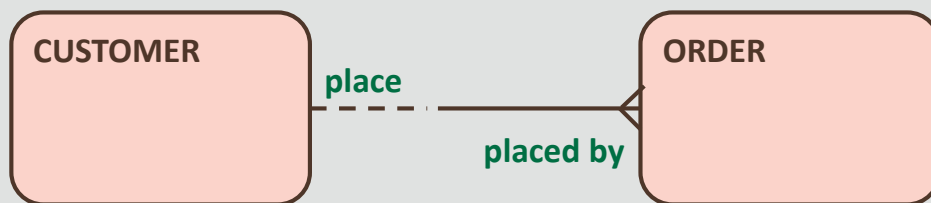


Optionality and Cardinality: Examples

- Each EMPLOYEE must hold one and only one JOB
- Each JOB may be held by one or more EMPLOYEE
- Each PRODUCT must be classified by one and only one PRODUCT TYPE
- Each PRODUCT TYPE may classify one or more PRODUCTS

Relationship Types

- All relationships represent the information requirements and the rules of the business
 - Many-to-one (M:1) or one-to-many (1:M)
 - Many-to-many (M:M)
 - One-to-one (1:1)



Example of a 1:M relationship

There are three types of relationships:

- **Many to one (M:1) or one to many (1:M):** There are crow's feet on one side of the relationship. The direction of the crow's feet determines whether the relationship is M:1 or 1:M. This type of relationship is the most common.
- **Many to many (M:M):** There are crow's feet on both sides of this relationship. It is common to see M:M relationships in a high-level ERD at the beginning of a project.
- **One to one (1:1):** This type of relationship is a line without crow's feet on either end. These types of relationships are rare.

Note: The notation is slightly different in Oracle SQL Developer Data Modeler, where one to many is 1:N and many to many is M:N.

Many-to-One and One-to-Many Relationships

- Many-to-one and one-to-many relationships (M:1 and 1:M) have cardinality of one or more in one direction and one and only one in the other direction

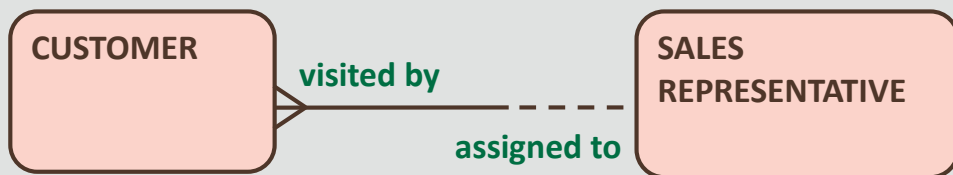


In the one- to- many (1:M) relationship, a single record in an entity (SALES REPRESENTATIVE) can be related to one or more records in another entity (CUSTOMER), but a single record in CUSTOMER can be related to only to one SALES REPRESENTATIVE.

Many-to-One and One-to-Many Relationships

- Business rules:

- Each CUSTOMER must be visited by one and only one SALES REPRESENTATIVE
- Each SALES REPRESENTATIVE may be assigned to one or more CUSTOMER



Many-to-Many Relationships

- Many-to-many relationships (M:M) have cardinality of one or more in both directions



- Business rules:
 - Each EMPLOYEE may be assigned to one or more JOB
 - Each JOB may be carried out by one or more EMPLOYEE

In the many- to- many (M:M) relationship, many records in one entity match many records in another entity.

One-to-One Relationships

- One-to-one relationships (1:1) have cardinality of one and only one in both directions



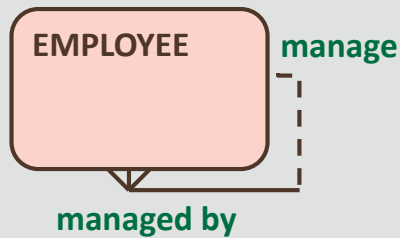
- Business rules:
 - Each COMPUTER must contain one and only one MOTHERBOARD
 - Each MOTHERBOARD must be contained in one and only one COMPUTER

The slide example is a one-to-one relationship because the cardinality is a line with no crow's feet in either direction.

These types of relations are the least common because they may instead be one entity with attributes contained in that entity.

Recursive Relationships

- A recursive relationship is a relationship with an entity and itself



- Business rules:
 - Each EMPLOYEE may manage one or more EMPLOYEE
 - Each EMPLOYEE must be managed by one and only one EMPLOYEE

Relationship Matrix: Collecting Information

- A relationship matrix can be used to collect initial information about the relationships among a set of entities

	CUSTOMER	ITEM	ORDER	WAREHOUSE
CUSTOMER			place	
ITEM			contained on	stored in
ORDER	placed by	issued for		
WAREHOUSE		store		

Relationship Matrix: Collecting Information

- A relationship matrix has the following characteristics:
 - A relationship matrix shows if and how each row entity on the left side of the matrix is related to each column entity shown across the top of the matrix
 - All entities are listed along the left side of the matrix and at the top of the matrix
 - If a row entity is related to a column entity, the name of that relationship is shown in the intersection box
 - If a row entity is not related to a column entity, the intersection box is empty
 - Each relationship above the diagonal line is the inverse or mirror image of a relationship below the line
 - Recursive relationships are represented by the boxes on the diagonal

Relationship Matrix: Mapping the Contents

- Map the contents of the relationship matrix to an ERD.

	CUSTOMER	ITEM	ORDER	WAREHOUSE
CUSTOMER			place	
ITEM			contained on	stored in
ORDER	placed by	issued for		
WAREHOUSE		store		

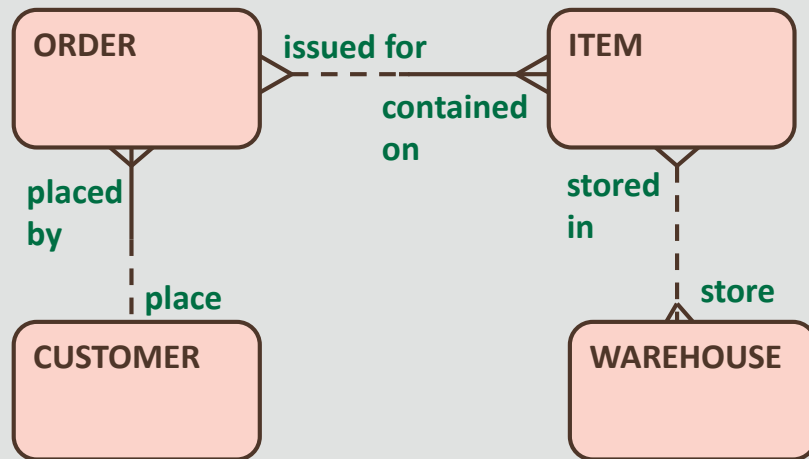
To map the matrix to an ERD, you draw a box for each entity and then draw the relationship and state the business rule. Doing so helps to determine each relationship's type and cardinality.

To help in the transformation between the matrix and the ERD, follow these steps:

- Determine a relationship's existence
- Name the relationship
- Determine the relationship's cardinality

Relationship Matrix: Mapping the Contents

- Results of mapping the contents of the relationship matrix to an ERD:



Determining a Relationship's Existence

- Examine each pair of entities to determine whether a relationship exists

	ACTIVITY	DEPARTMENT	EMPLOYEE
ACTIVITY			
DEPARTMENT			
EMPLOYEE			

The first step in creating a relationship is to determine its existence. Ask yourself, “Does a significant relationship exist between ENTITY A and ENTITY B?”

In the slide example, consider the following questions:

- Is there a significant relationship between DEPARTMENT and EMPLOYEE? The answer is yes.
- Is there a significant relationship between DEPARTMENT and ACTIVITY? The answer is no.
- Is there a significant relationship between ACTIVITY and EMPLOYEE? The answer is yes.

Log the relationships among ACTIVITY, DEPARTMENT, and EMPLOYEE on a relationship matrix. The check marks indicate that a relationship exists.

A relationship matrix is used to systematically examine each pair of entities.

Determining a Relationship's Existence

- Examine each pair of entities to determine whether a relationship exists

	ACTIVITY	DEPARTMENT	EMPLOYEE
ACTIVITY			✓
DEPARTMENT			✓
EMPLOYEE	✓	✓	

The first step in creating a relationship is to determine its existence. Ask yourself, “Does a significant relationship exist between ENTITY A and ENTITY B?”

In the slide example, consider the following questions:

- Is there a significant relationship between DEPARTMENT and EMPLOYEE? The answer is yes.
- Is there a significant relationship between DEPARTMENT and ACTIVITY? The answer is no.
- Is there a significant relationship between ACTIVITY and EMPLOYEE? The answer is yes.

Log the relationships among ACTIVITY, DEPARTMENT, and EMPLOYEE on a relationship matrix. The check marks indicate that a relationship exists.

A relationship matrix is used to systematically examine each pair of entities.

Naming the Relationship

- Name each direction of a relationship

	ACTIVITY	DEPARTMENT	EMPLOYEE
ACTIVITY			assigned to
DEPARTMENT			contain
EMPLOYEE	participate in	assigned to	

Relationship names represent a role and tend to be passive verbs, noun role names, or prepositions. Try not to use "related to" or "associated with" as relationship names because they lack specific meaning and are weak.

The following is a sample list of relationship name pairs to assist you in naming relationships:

- based on/the basis for
- bought from/the supplier of
- description of/for
- operated by/the operator for
- represented by/the representation of
- responsible for/the responsibility of

Log the relationship names in the relationship matrix as shown in the slide.**

Project Exercise 4

- DFo_2_5_4_Project
 - Oracle Baseball League Store Database
 - Using a Relationship Matrix



Summary

- In this lesson, you should have learned how to :
 - Define and recognize examples of relationships and corresponding foreign keys
 - Identify the optionality of relationships
 - Identify the cardinality of relationships
 - Types of relationships
 - Relationship matrix



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Foundations

2-6

Entity Relationship Modeling (ERDs)

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Describe data modeling
 - Explain "implementation-free" as it relates to data models and database design implementation
 - List the four goals of entity relationship modeling
 - Identify an entity relationship diagram (ERD)
 - Map relationships using ERDish
 - Construct ERD components that represent entities and attributes according to diagramming conventions



Purpose of Conceptual Modeling

- When you design a house, you eventually want to see the house built
- Even if you don't do the actual construction, you need to understand the builder's terms to help them take your conceptual design and make it a physical reality
- The database conceptual model can be used for further discussion between designers, database administrators, and application developers

The architect is trained in the skills of translating ideas into models. The architect listens to the description of the ideas and asks many questions that are then put into a diagram (the blueprint) that allows for discussion and analysis, giving advice, describing sensible options, documenting it, and confirming it with the future homeowners. This diagram provides the future homeowners with a plan of the home they want.

A building contractor needs the blueprint of the house with an exact description of the materials to be used, the size of the roof beams, the capacity of the plumbing, and many other specifications. The builder follows the plan and has the knowledge to construct what is in the blueprint into a physical reality.

Purpose of Conceptual Modeling

- A conceptual model is important to a business because it:
 - Describes the exact information needs of the business
 - Facilitates discussion
 - Prevents mistakes and misunderstandings
 - Forms a sound basis for physical database design
 - Documents the processes (also known as business rules) of the business
 - Takes into account regulations and laws governing this industry

Conceptual Modeling

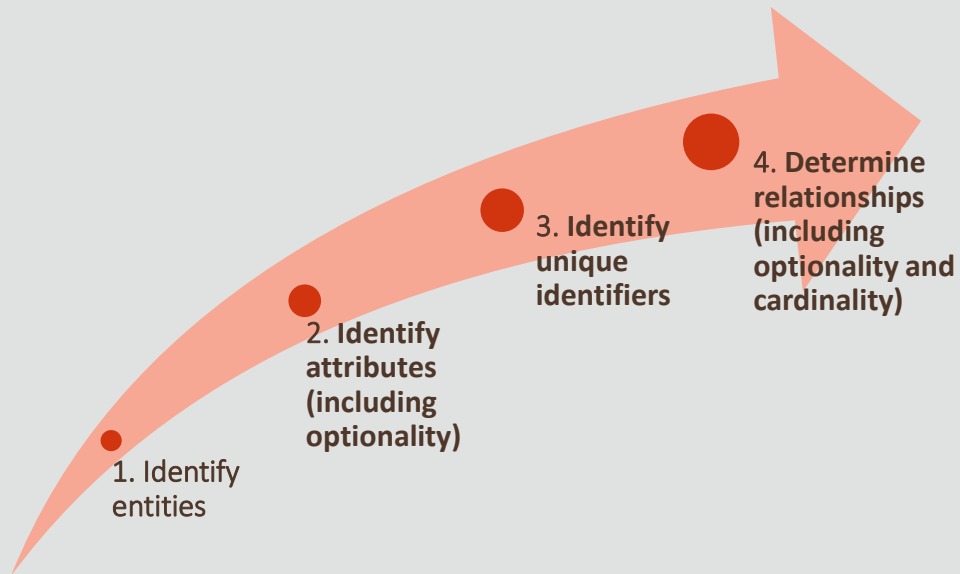
- A conceptual model is a formal model which:
 - Describes the things of significance to an organization (entities)
 - Identifies the highest-level relationships between the different entities, but may or may not include cardinality and nullability
 - Does not specify the attributes or the unique identifier for each entity

The conceptual modeling has a formal analysis and design method that uses a set of guidelines and rules to capture the semantics of a domain. Formal methods include textual or graphical notations to create, present, validate, and manipulate data models. It clarifies identification of entities, and relationships. It provides a basis for discussion and refinement.

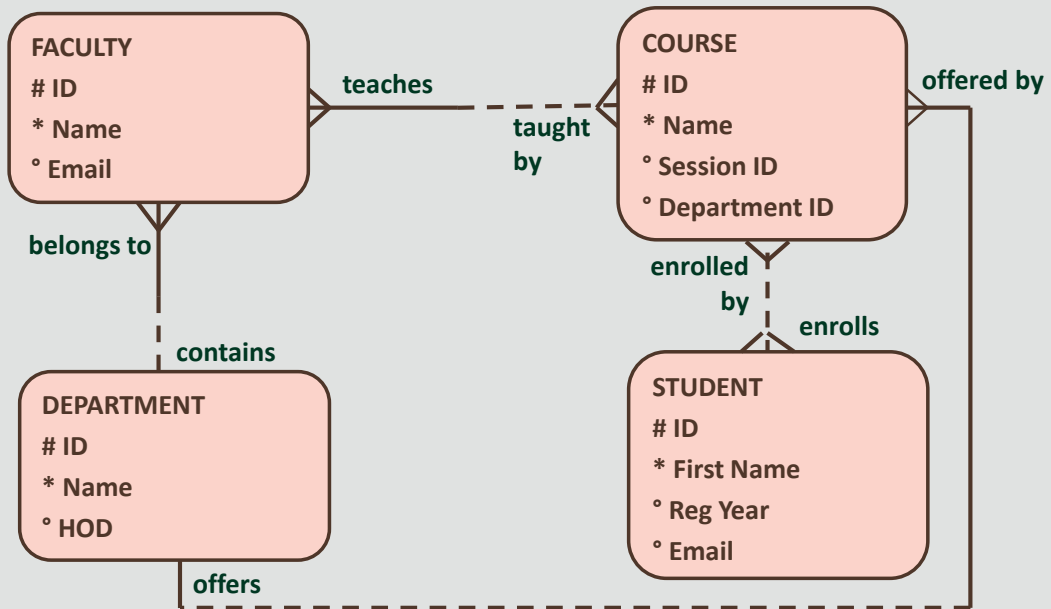
Logical Modeling

- The logical data model :
 - Describes the data in as much detail as possible, without regard to how it will be physically implemented in the database
 - Is normally derived from a conceptual data model
 - Includes all entities, attributes, UIDs and relationships as well as optionality and cardinality of these items
- The logical model is illustrated using an ERD

Steps to Create a Logical Model



Logical Modeling: Example

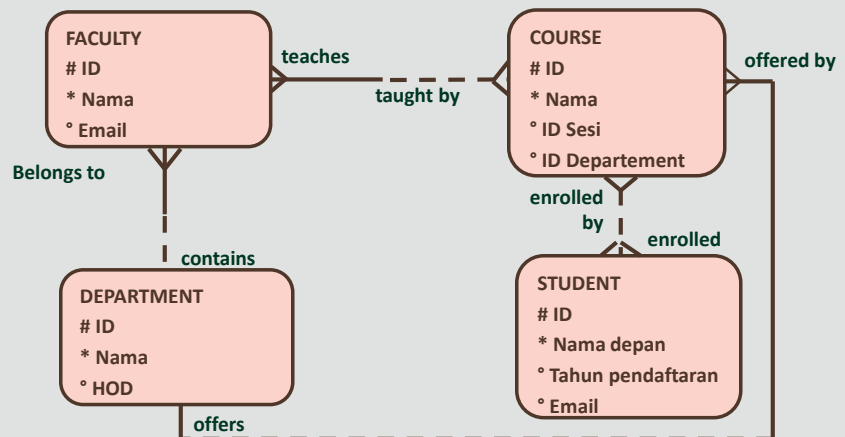


Entity Relationship Diagram (ERD)

- An ERD is a model that identifies the concepts or entities that exist in a system and the relationships between those entities
- It serves several purposes:
 - The database analyst/designer gains a better understanding of the information to be contained in the database through the process of constructing the ERD
 - It serves as a documentation tool
 - It is used to communicate the logical structure of the database to users. In particular, it effectively communicates the logic of the database to users

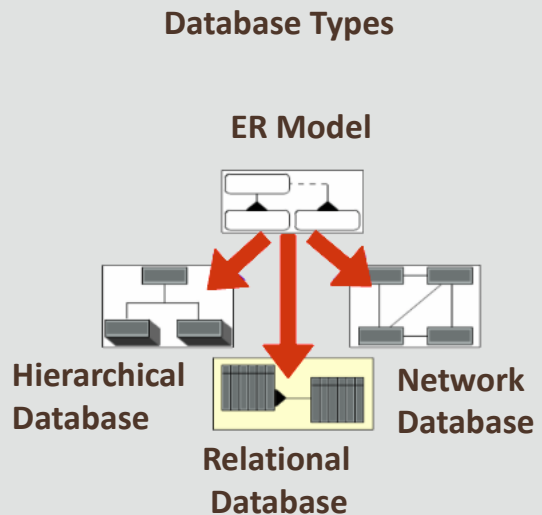
Entity Relationship Diagram (ERD)

- An ERD can be used to represent the data requirements of a business, regardless of the type of database that is used, and even in the absence of one
- A graphical representation of entities and their relationships to each other



Implementation-Free Models

- A good logical data model stays the same regardless of the type of database system that is eventually built—or implemented—on
- This is what “implementation-free” model means



The data model should stay the same even if a database is not used at all; for example, when the data is eventually stored on pieces of paper in a filing cabinet.

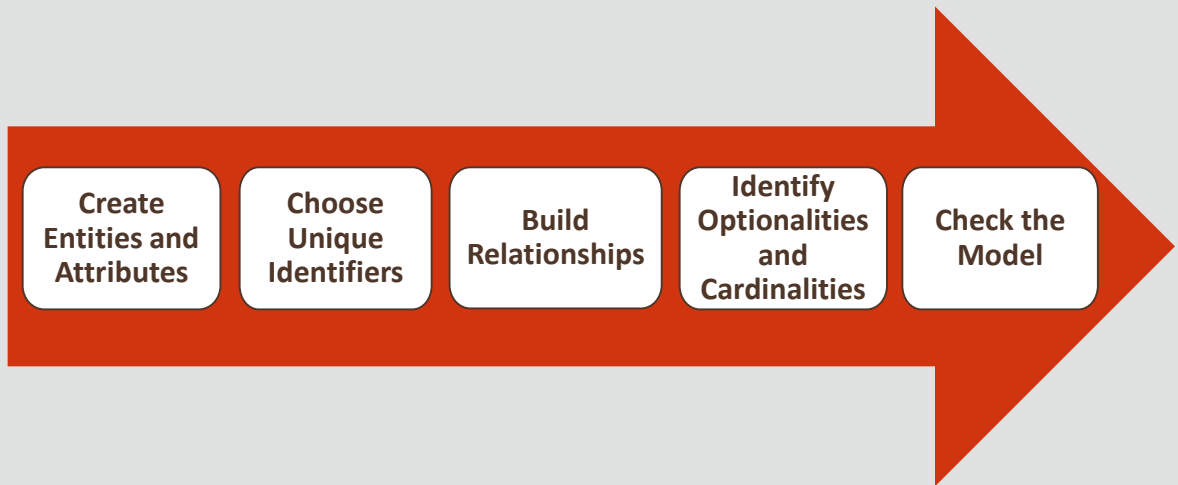
Entity Relationship Model (ERM)

- Is derived from business specifications, and its intended goal is to create a clear picture of the information that will be stored in a future database
- Is a list of all entities and attributes as well as all relationships between the entities that are of importance
- Provides background information such as entity descriptions, data types, and constraints
- Does not require a diagram, but the diagram is typically a very useful tool

Goals of ER Modeling

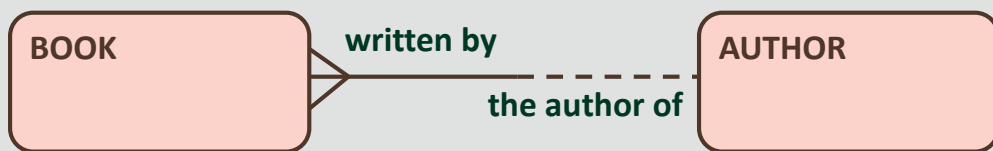
- Capture all required information
- Ensure that information appears only once
- Model no information that is derivable from other information that is already modeled
- Locate information in a predictable, logical place

Steps to Build an ERD



Create ERDish sentences to represent ERDs

- ERDish is the vocabulary used to clearly communicate the business rules that are captured on an ERD
- Use ERDish language to state relationships between entities in an ERD
- Simply break down each ERDish sentence into its components



Data modeling uses industry-specific terminology as well, which we will call ERDish for the purposes of this class. It will give you a common language both when collecting the business rules from your client and communicating them to the Database Administrators who will implement your design.

You have already been speaking and writing it, when you identified relationships and specified optionality and cardinality.

Components of ERDish

- EACH
- Entity A
- OPTIONALITY (must be/may be)
- RELATIONSHIP NAME
- CARDINALITY (one and only one/ one or more)
- Entity B

ERDish Example

Because a relationship has two sides, first read one side from left to right.



1. EACH
2. **BOOK** (entity A)
3. **MUST BE** (optionality, solid line)
4. **WRITTEN BY**(relationship name)
5. **ONE (AND ONLY ONE)** (cardinality, single toe)
6. **AUTHOR**(entity B)



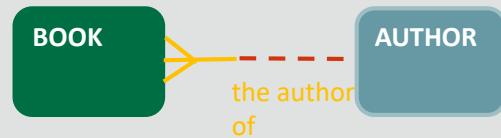
1. EACH
2. **AUTHOR** (entity B)
3. **MAY BE** (optionality, dotted line)
4. **THE AUTHOR OF**(relationship name)
5. **ONE OR MORE** (cardinality, crow's foot)
6. **BOOK** (entity A)

Next, read the relationship from right to left.

Case Scenario



1. EACH
2. **BOOK** (entity A)
3. **MUST BE** (optionality, solid line)
4. **WRITTEN BY**(relationship name)
5. **ONE AND ONLY ONE** (cardinality, single toe)
6. **AUTHOR**(entity B)

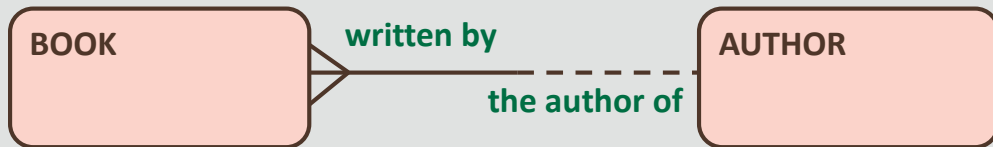


1. EACH
2. **AUTHOR** (entity B)
3. **MAY BE** (optionality, dotted line)
4. **THE AUTHOR OF**(relationship name)
5. **ONE OR MORE** (cardinality, crow's foot)
6. **BOOK** (entity A)



Validating the Relationship

- Reexamine the ERD and validate the relationship



- Each **BOOK** must be written by one and only one **AUTHOR**
- Each **AUTHOR** may be the author of one or more **BOOK**

Sporting Goods Business Scenario

- I'm a manager of a sporting goods wholesale company that operates worldwide to fill orders of retail sporting goods stores. The stores are our customers (some of our people prefer to call them our clients)
- Right now we have fifteen customers worldwide, but we're trying to expand our customer base by about 10% each year starting this year

Sporting Goods Business Scenario

- Our two biggest customers are in the United States: Big John's Sports Emporium in San Francisco, California, and Women's Sports in Seattle, Washington
- For each customer, we must track an ID and a name. We may also track an address (including the city, state, zip code, and country) and a phone number
- We maintain warehouses in different regions to fill our customer orders

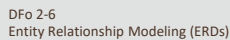
Sporting Goods Business Scenario

- For each order, we must track an ID. We may also track the date ordered, date shipped, and payment type if the information is available. Our order entry personnel are well versed in our product line.
- We hold frequent meetings with Marketing to learn about new products. The result is better customer satisfaction because we can answer customer questions.

Sporting Goods Business Scenario

- We deal with a few select customers and maintain a specialty product line. For each product, we must know the ID and name. Occasionally we must also know the description, suggested price, and unit of sale
- When necessary we also want to be able to track very long descriptions of our products and pictures of our products

ORACLE
Academy



25

Project Exercise

- DFo_2_6_Project
 - Oracle Baseball League Store Database
 - Entity Relationship Modeling



ORACLE
Academy

DFo 2-6
Entity Relationship Modeling (ERDs)

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

26

Summary

- In this lesson, you should have learned how to:
 - Describe data modeling
 - Explain "implementation-free" as it relates to data models and database design implementation
 - List the four goals of entity relationship modeling
 - Identify an ERD
 - Map relationships using ERDish
 - Construct ERD components that represent entities and attributes according to diagramming conventions



The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy